Group 33 | LogiEdge Final Report

FreightBridge Logistics Pvt. Ltd. — Offline-first refrigerated-fleet edge AI

Five-section consolidated report | Approximately 2,543 body words | Evidence: Phase 1 and Phase 2 reports plus verified execution logs

1. FreightBridge Deployment Context

FreightBridge Logistics requires refrigerated trucks to detect temperature and refrigeration-unit abnormalities even when cellular coverage is intermittent. The LogiBridge implementation addresses this through local sensor ingestion, windowed feature extraction, a three-class edge classifier, local alerts and optional upstream telemetry. The pilot covers 85 trucks and the proposed fleet expansion covers 265. The operational objective is to produce a useful fault alert within 90 seconds. This is an offline-first design: inference and immediate warning do not depend on a round trip to a cloud service.

Four constraints drive the design. First, connectivity is unreliable on moving routes, so sensor acquisition, inference and alert generation must continue offline; the cloud is for synchronization, fleet-wide analysis and managed updates rather than the immediate safety decision. Second, the AI electronics must fit a 10 W power budget on the vehicle's 12 V supply. Third, the hardware must withstand vehicle vibration, heat, electrical transients and prolonged operation. Fourth, cost must scale across the pilot and the larger fleet without consuming the benefit of reduced spoilage. A further functional constraint is the 90-second detection deadline.

The hardware comparison in Phase 1 recommends a Raspberry Pi 5 (8 GB) with AI HAT+ (13 TOPS). Using the report's assumed 90% DC-DC conversion efficiency, its 7.5 W device consumption implies 8.33 W at the 12 V rail, leaving only 1.67 W of a 10 W rail-side budget. At an indicative Rs 15,000 per truck, hardware totals Rs 12.75 lakh for 85 vehicles and Rs 39.75 lakh for 265. The Jetson Orin Nano alternative is priced at Rs 45,000 and assumed to draw 15 W, exceeding the specified cap. The STM32H7 MCU is cheaper and lower-power, but lacks the Linux ecosystem needed for the selected broker, container, log and update architecture without additional firmware engineering. These are planning assumptions, not physical measurements on installed trucks.

Power is therefore the dominant vertex of the performance-power-cost Constraint Triangle: the latency requirement is comparatively easy for a small classifier, while a hard power violation cannot be compensated by faster inference. The recommended Pi also offers software flexibility, but its narrow power margin requires a real measurement including storage, modem, sensors and accelerator. Automotive-grade power conditioning, a robust enclosure and industrial storage are deployment prerequisites.

The logical architecture is sensor streams to an on-truck MQTT broker, then a preprocessing/windowing service, then TFLite inference, a local alert log and a synchronization path to fleet systems. Temperature is sampled at 1 Hz and vibration at 0.5 Hz; door events are available as context but are not part of the six-feature classifier. MQTT decouples publishers from the model service, while the local inference path remains operational when the WAN is unavailable. The Docker demonstration establishes the software path, not a fully ruggedized in-vehicle installation.

The Phase 1 Roofline exercise uses the assignment's separate reference workload: 45 MFLOP and 18 MB transferred per inference, a 16 GFLOP/s CPU ceiling and 12 GB/s memory bandwidth. Arithmetic intensity is 45/18 = 2.5 FLOP/byte; the ridge point is 16/12 = 1.33 FLOP/byte. The reference point is compute-bound, with idealized compute and memory lower bounds of 2.81 ms and 1.50 ms respectively. This explains why reducing operations through pruning may help, while INT8 can reduce both arithmetic and memory demand. These Roofline inputs are illustrative and must not be confused with measurements of the much smaller deployed 6-32-16-3 MLP.

2. Sensor Pipeline and MQTT Design

The simulator provides reproducible Normal, temperature-drift, vibration and combined-anomaly scenarios. It publishes temperature and vibration to truck-specific MQTT topics, such as truck/T001/temp and truck/T001/vib. The deployed inference service subscribes to both streams and publishes decisions to logibridge/trucks/T001/inference. A broker-local design allows consumers to restart independently and makes it possible to attach dashboards or alert relays without changing the classifier. The live demonstration used Mosquitto on port 1883; its anonymous-access demo configuration must not be carried into production. Production messaging should use authenticated clients, TLS, topic-level authorization and deliberate QoS policies: QoS 1 is appropriate for consequential alerts where duplicate handling is implemented, while high-rate telemetry can use QoS 0 if occasional loss is acceptable. These are deployment recommendations, not verified security settings of the demonstration.

Preprocessing applies a causal five-sample moving average separately to each stream. It then forms 30-second windows every 10 seconds, retaining the different sensor sampling rates: approximately 30 temperature readings and 15 vibration readings in a complete window. Missing or malformed samples are not silently fabricated; incomplete windows are skipped when coverage is insufficient. Filtering occurs before window feature extraction, ensuring that training and inference use the same signal transformation.

The six features are temperature mean, temperature standard deviation, temperature slope in degrees Celsius per minute, vibration RMS, vibration peak magnitude and vibration excess kurtosis. The first three summarize temperature level, variability and trend; the last three summarize vibration energy, transient amplitude and distribution shape. Feature-level fusion concatenates these into one vector, permitting a compact MLP to learn interactions between temperature drift and mechanical vibration rather than issuing unrelated single-sensor alarms.

Normalisation uses frozen Normal-class training statistics saved in training_stats.npy. The Phase 2 report documents a 2-by-6 mean-and-standard-deviation array computed from 57 windows of ten minutes of clean simulation. Runtime inference loads these values; it does not refit them on the truck. This separation is essential because recomputing a mean during an anomaly can erase the very deviation the detector is intended to recognize.

The mandatory three-sigma sensitivity experiment was performed with a separate 6-16-3 reference classifier, trained with five seeds. Using the correct saved statistics yielded 99.67% mean accuracy. Shifting the stored feature means by +3 standard deviations reduced accuracy to 98.38%, a 1.29-point decline; shifting by -3 standard deviations reduced it to 91.28%, an 8.38-point decline. Recomputing statistics from each live test run produced only 43.60%, approximately the majority-class baseline. This asymmetric response matters operationally: sensor calibration offsets can increase missed early warnings in one direction and false alarms in the other. The experiment is evidence of normalisation sensitivity, not a measurement of the final three TFLite variants.

The main training dataset comprises 291 labelled, overlapping windows: 117 Normal, 87 Warning and 87 Critical. The model is a 6-32-16-3 multilayer perceptron with ReLU hidden units and a softmax output. The original stratified validation split scored 56/57, or 98.25%, exceeding the 88% requirement. Additional checks highlighted the risk of overly optimistic random splits because neighbouring windows share data. Augmenting training with faults that begin mid-run improved the report's independent onset checks. This is particularly relevant to refrigerated fleets: an alarm system must detect a fault beginning after a long normal journey, not only a fault present from the first sample.

A remaining safety limitation is the difference between simulator-mode labels and temperature-threshold severity. The Phase 2 report notes that prolonged temperature drift can exceed the Critical threshold while retaining a Warning label from its generation mode. High accuracy against those labels does not prove compliance with a threshold-based clinical or pharmaceutical escalation policy. A production release should reconcile labels with operating thresholds and test Critical recall on genuinely independent trips.

3. Model Deployment and Pipeline Mapping

The implemented workflow maps onto ten practical ML pipeline stages: (1) problem and operating-constraint definition; (2) sensor simulation and data collection; (3) validation and cleaning; (4) causal filtering and sliding windows; (5) six-feature extraction and frozen normalisation; (6) labelled dataset generation and splitting; (7) MLP training; (8) validation and fault-onset checks; (9) TFLite conversion, benchmarking and deployment; and (10) monitoring, feedback and update. The ordering matters because a change in feature definition or training statistics requires a compatible model update. Treating the entire pipeline as a versioned unit reduces train-serving skew.

The container uses a slim Python 3.11 base, pinned runtime dependencies and a non-root application user. The Dockerfile installs requirements before its final COPY model.tflite instruction. Consequently, when only the model artifact changes, the expensive base and dependency layers can remain cached. MODEL_PATH selects the runtime model file, and MQTT variables select the broker and truck identity. The software was demonstrated on Docker/WSL rather than on the proposed Raspberry Pi hardware; host timing and power figures should therefore not be projected directly onto the truck.

The live demonstration established a complete path from the simulator through Mosquitto to TFLite inference. The service logged Normal predictions before anomaly onset and Critical predictions afterward. Ansible subsequently deployed localhost:5000/logiedge/inference:v1, loaded the expected model checksum prefix 326543ea93a3, enabled PSI monitoring and connected to localhost:1883. The deployed container used host networking, explaining why localhost correctly reached the broker in this particular setup.

For OTA validation, a corrected model-only experiment compared an M1 FP32 image with a genuinely different M3 pruned INT8 model. The Docker layer comparison showed ten layers in each image, nine identical and one changed. The demonstration estimated 6,315.02 MB of fleet transfer avoided for 85 trucks, or 99.98%, with an illustrative Rs 631.50 saving at the specified data rate. These figures are an image-layer transfer model, not observed cellular usage or a successful rollout to remote trucks. A local registry served the image for the Ansible demonstration.

A full-replacement strategy is operationally simple but exposes all trucks simultaneously to a faulty model. Canary rollout deploys to a small subset first and checks alert volume, drift, crash rates and Critical recall before expanding. Shadow mode runs a candidate alongside the current model without allowing the candidate to control alarms; it supplies comparative evidence at additional compute and energy cost. For the 85-truck pilot, canary is the preferred default after offline validation, with shadow evaluation for changes affecting severity rules. The layer-cache benefit applies to model-only image updates regardless of the rollout strategy; its magnitude depends on the real registry, compression, cached layers and network conditions.

An update must be signed or otherwise authenticated, tied to a compatible preprocessing-statistics version and reversible. A failed rollout should restore the prior image and reference distribution, not merely the model weights. Those safeguards remain architectural recommendations; the local OTA demonstration verified caching rather than production-grade signing, remote orchestration or rollback.

4. Optimisation and Pareto Analysis

Three TensorFlow Lite artifacts were generated and benchmarked: M1 FP32, M2 post-training INT8 and M3 structured-pruned INT8. M1 is the full-precision baseline, M2 changes numerical representation through post-training quantisation, and M3 additionally removes hidden units. The M3 implementation used a NumPy structured-pruning routine with a polynomial-decay schedule, reducing hidden widths from 32 and 16 to 21 and 10 before INT8 conversion. It did not execute TensorFlow Model Optimization's UpdatePruningStep callback, which is an important tooling-compliance distinction if the assignment mandates that API.

The measured artifact sizes were 5,100 bytes for M1, 3,200 for M2 and 2,768 for M3. Relative to M1, M2 is about 37.3% smaller and M3 about 45.7% smaller. In the WSL benchmark, mean inference latencies were 0.0018 ms,

0.0164 ms and 0.0120 ms respectively. The 95th-percentile figures were 0.0020 ms, 0.0294 ms and 0.0130 ms. All three variants achieved 100% accuracy and 100% Critical recall on the benchmark's validation sample. This result is encouraging but should not be read as a guaranteed production recall, especially given overlapping simulated windows and the label-severity issue noted earlier.

The Pareto trade-off is counterintuitive: FP32 was fastest on the measured host despite being the largest file. Quantisation can introduce overhead when a particular CPU runtime or delegate lacks an optimized execution path for the small network. M3 is the smallest, and it is faster than M2 in the recorded run, but neither INT8 variant beat M1 on host latency. Therefore, the report should not claim that INT8 universally accelerates inference. The measured results, rather than theoretical speedups, govern the deployment decision.

The benchmark estimated energy per inference at 0.00118 mJ for M1, 0.01131 mJ for M2 and 0.00829 mJ for M3 using a 15 W TDP-based method. These are derived estimates, not power-meter measurements; CPU utilization was approximately 4.5–4.6% in the benchmark. A Raspberry Pi power meter and representative duty cycle are required before claiming fleet energy savings.

For the current tiny workload, M1 is the pragmatic latency baseline and has the strongest direct runtime demonstration. M3 is attractive when storage, transmission size or future model count dominates, provided its runtime and class-specific behavior are revalidated on the actual edge target. A responsible recommendation is to retain M1 as the pilot reference, qualify M3 in a controlled canary, and decide from on-device measurements of latency, energy, stability and severity-aligned recall. The supplied Pareto chart and raw CSV should accompany this analysis so the comparison remains auditable.

5. MLOps Monitoring and Reflection

The monitoring demonstration used Population Stability Index (PSI) to compare the distribution of Normal-class confidence against a reference built from clean simulated windows. It retained a rolling window of 100 inferences, evaluated drift at a regular interval, raised an alert above PSI 0.25 and used a lower recovery threshold of 0.10 to avoid oscillating alerts. In the scripted scenario the anomaly began at simulated time 20:00, the first alert appeared at 21:30 with PSI 0.387, and the peak reached 2.217. After clean behavior resumed at 25:00, the monitor fell below recovery threshold at 41:30 and reached zero by 42:30. These are simulated-time results, not elapsed minutes from a field trial.

A notable implementation insight was that maximum softmax confidence remained high even for confident wrong-class predictions, making it a poor drift indicator in this demonstration. Normal-class confidence was more informative because it changed as anomalies displaced the model's normal operating distribution. PSI nevertheless detects distribution changes, not their cause: sensor replacement, route changes, genuine faults and software mismatches may all trigger it. Operational response should combine PSI with data-quality checks and actual alert outcomes rather than automatically retraining on every shift.

The Ansible playbook defined seven deployment tasks and was executed twice without intervening changes. The recorded first run completed with six successful tasks, one conditional skip, one changed task and no failures. The second run again had six successful tasks and one skip, but changed=0. This verifies local idempotency under the tested state. The deployed container remained running, loaded its TFLite model and subscribed to the MQTT sensor topics. All 213 automated tests subsequently passed.

The hardest engineering issue was aligning optimization, packaging and deployment assumptions: the original pruning callback path did not run as expected, the OTA demonstration initially compared identical model bytes, and Ansible required registry setup and a WSL-compatible privilege approach. Correcting these exposed the importance of checking actual artifacts and logs rather than trusting a script's success message alone. One architectural improvement would be a versioned, signed model bundle containing the TFLite file, preprocessing statistics, class-threshold policy and PSI reference, deployed atomically with rollback. This would reduce the risk of

incompatible partial updates.

Overall, LogiBridge demonstrates an offline-capable software pipeline and repeatable local deployment evidence. Its principal limitations are simulated training data, host-only benchmarks, the unverified automotive power envelope, non-production MQTT security and unresolved severity-label alignment. The next engineering gate should be a Raspberry Pi pilot with independent trip data, measured rail power, authenticated MQTT, tested rollback and severity-based Critical recall before broader fleet rollout.

Source evidence and verification scope

Phase 1 Report: hardware choice, Constraint Triangle and Roofline analysis. Phase 2 Report: preprocessing experiment, training, deployment design and monitoring. Subsequent verified execution: three TFLite variants and benchmark CSV, MQTT container logs, drift demo, corrected OTA layer comparison, Ansible two-run recap and 213-test result. Values from the reports are distinguished from estimates and later runtime evidence in the narrative.